

✓ 07. 트랜스포머

트랜스포머: 신경망 아키텍처 중 하나

- 주요 기능: 순차적 방식이 아닌 병렬로 입력 시퀀스를 처리
- 순차 처리나 반복 연결에 의존하지 않고 입력 토큰 간의 관계를 직접 처리하고 이해할 수 있도록 하는 셀프 어텐션 기반
 - 재귀나 합성곱 연산 없이 입력 토큰 간의 관계 직접 모델링 가능
- 대용량 데이터셋에서 매우 효율적, 장기적인 종속성을 포함하는 작업에 주로 사용

오토 인코딩: 랜덤하게 문장의 일부를 빈칸 토큰으로 만들고 해당 빈칸에 어떤 단어가 적절한지 예측

- 예측되는 토큰의 양옆에 있는 토큰들을 참조하는 양방향 구조를 가짐(인코더)

자기 회귀 방식: 이전 단어들이 주어졌을 때 다음 단어가 무엇인지 맞힘

- 예측되는 토큰의 왼쪽에 있는 토큰들만 참조하는 단방향 구조를 가짐(디코더)

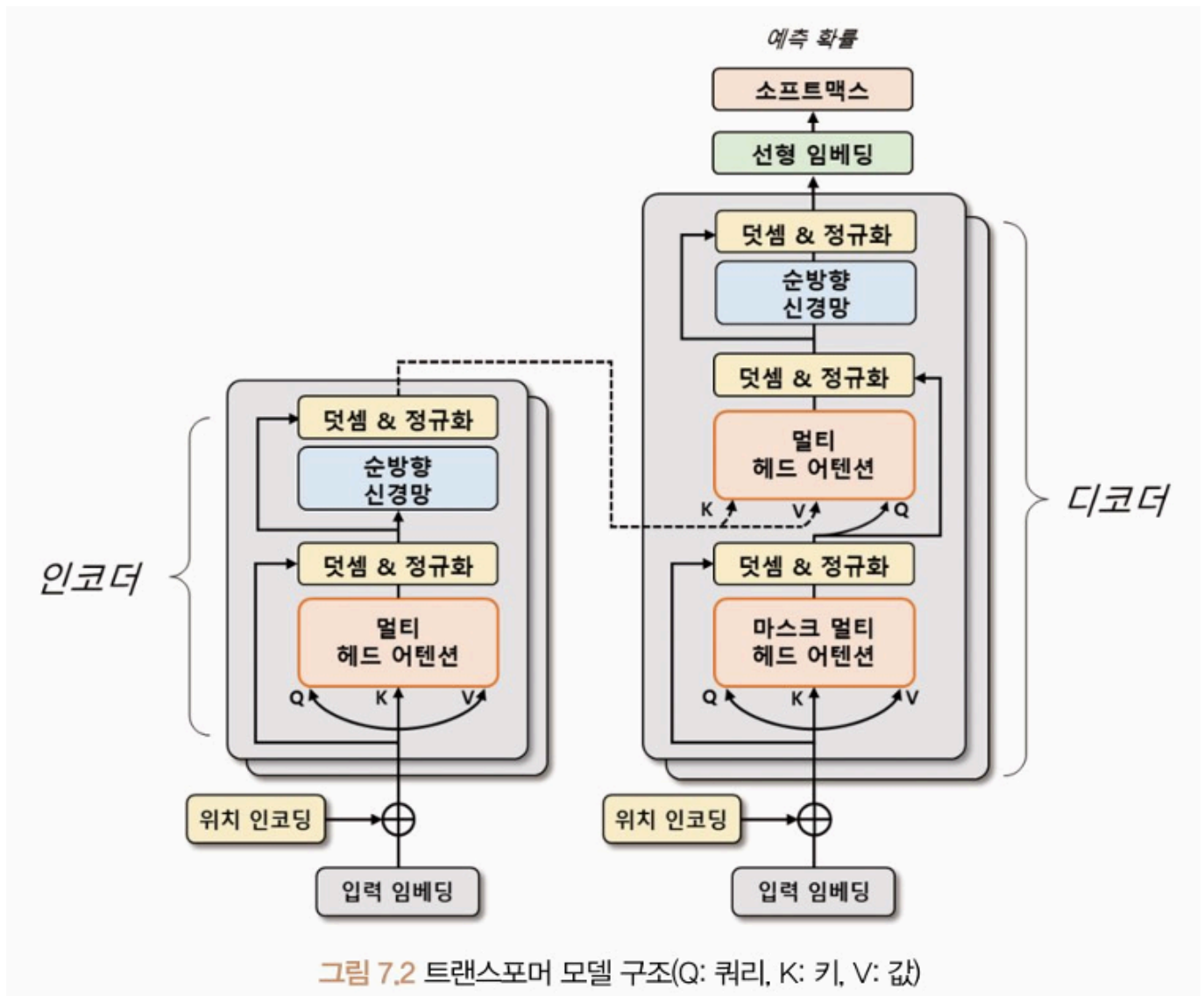
표 7.1 트랜스포머 모델 구조

모델	학습구조	학습방법	학습 방향성
BERT	인코더	오토 인코딩	양방향
GPT	디코더	자기 회귀	단방향
BART	인코더+디코더	오토 인코딩+자기 회귀	양방향+단방향
ELECTRA	인코더+판별기	오토 인코딩+대체 토큰 탐지	양방향
T5	인코더+디코더	오토 인코딩+자기 회귀+다양한 자연어 처리 작업을 학습	양방향

✓ Transformer

트랜스포머: 다양한 자연어 처리 분야에서 많은 성과를 내는 모델

- 순환 신경망이나 합성곱 신경망과 달리 어텐션 메커니즘만을 사용하여 시퀀스 임베딩 표현
 - 인코더와 디코더 간의 상호작용으로 입력 시퀀스의 중요한 부분에 초점을 맞추어 문맥 이해, 적절한 출력 생성
 - 인코더: 입력 시퀀스를 임베딩하여 고차원 벡터로 변환
 - 디코더: 인코더의 출력을 입력으로 받아 출력 시퀀스를 생성
- 기존 순환 신경망 기반 모델보다 학습 속도 빠름, 병렬 처리 가능
- 임베딩 과정에서 문장 전체 정보를 고려, 문장의 길이가 길어져도 성능 유지



- 트랜스포머 모델 구조
 - 인코더와 디코더 각각 N 개의 트랜스포머 블록으로 구성
 - 블록은 멀티 헤드 어텐션과 순방향 신경망으로 구성
 - 멀티 헤드 어텐션: 입력 시퀀스에서 쿼리, 키, 값 벡터를 정의해 입력 시퀀스들의 관계를 셀프 어텐션하는 벡터 표현 방법
 - 쿼리와 각 키의 유사도 계산, 유사도를 가중치로 사용하여 값 벡터 합산
 - 어텐션 행렬은 입력 시퀀스 각 단어의 임베딩 벡터를 대체
 - 순방향 신경망: 임베딩 벡터를 더욱 고도화하기 위해 사용
 - 여러 개의 선형 계층으로 구성
 - 입력 벡터에 가중치를 곱하고, 편향을 더하며, 활성화 함수를 적용
- 입력 시퀀스 데이터를 소스와 타겟 데이터로 나눠 처리
 - 인코더는 소스 시퀀스 데이터를 위치 인코딩된 입력 임베딩으로 표현, 출력 벡터 생성
 - 출력 벡터는 입력 시퀀스 데이터의 관계를 잘 표현할 수 있게 구성

- 디코더는 마스크 멀티 헤드 어텐션을 사용해 타깃 시퀀스 데이터를 순차적으로 생성
- 최종적으로 생성된 디코더 출력 벡터는 선형 임베딩으로 재표현

입력 임베딩과 위치 인코딩

- 위치 인코딩: 입력 시퀀스의 순서 정보를 모델에 전달하는 방법
 - 위치 인코딩 벡터는 sin과 cos 함수를 사용해 생성
 - 임베딩 벡터와 위치 정보가 결합된 최종 입력 벡터 생성
 - 각 토큰의 위치를 각도로 표현해 sin과 cos 함수로 위치 인코딩 벡터 계산
 - 트랜스포머 모델에서 입력 시퀀스의 순서 정보를 보존하기 위한 중요한 방법 중 하나

수식 7.1 위치 인코딩

$$PE(pos, 2i) = \sin(pos/10000^{2i/d_{model}})$$

$$PE(pos, 2i + 1) = \cos(pos/10000^{2i/d_{model}})$$

- pos: 입력 시퀀스에서 해당 단어의 위치
- i: 임베딩 벡터의 차원 인덱스
 - 짝수면 sin 함수 수식 / 홀수면 cos 함수 수식 적용

특수 토큰: 입력 시퀀스의 시작과 끝을 나타내거나 마스킹 영역으로 사용

- 트랜스포머는 특수 토큰을 활용하여 문장을 표현
- 특수 토큰 예시
 - BOS: 문장의 시작 / EOS: 문장의 종료
 - UNK: 어휘 사전에 없는 단어, 모르는 단어 -> 이전에 본 적 없는 단어 처리
 - PAD: 모든 문장을 일정한 길이로 맞추기 위해 사용
- 특수 토큰을 통해 생성된 문장 토큰 배열을 어휘 사전에 등장하는 위치에 원-핫 인코딩으로 표현

트랜스포머 인코더: 입력 시퀀스를 받아 여러 개의 계층으로 구성된 인코더 계층을 거쳐 연산을 수행

- 위치 정보를 반영하기 위해 위치 임베딩 벡터를 입력 벡터에 더해줌

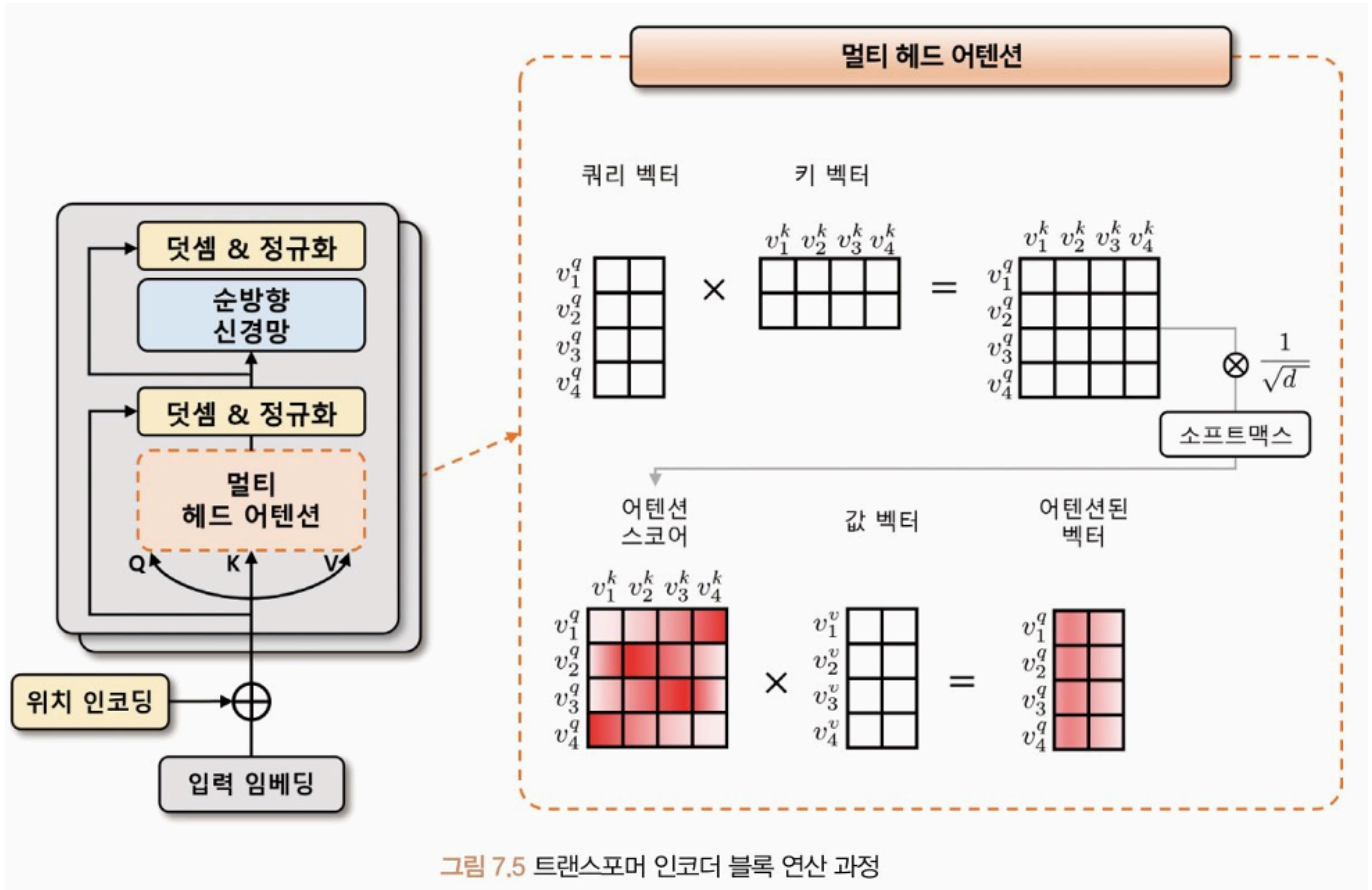


그림 7.5 트랜스포머 인코더 블록 연산 과정

- 위치 인코딩이 적용된 소스 데이터의 입력 임베딩을 입력받음
- 쿼리 벡터: 현재 시점에서 참고하고자 하는 정보의 위치를 나타내는 벡터
- 키 벡터: 쿼리 벡터와 비교되는 대상, 쿼리 벡터를 제외한 입력 시퀀스에서 탐색되는 벡터
- 값 벡터: 쿼리 벡터와 키 벡터로 생성된 어텐션 스코어를 얼마나 반영할지 설정하는 가중치 역할

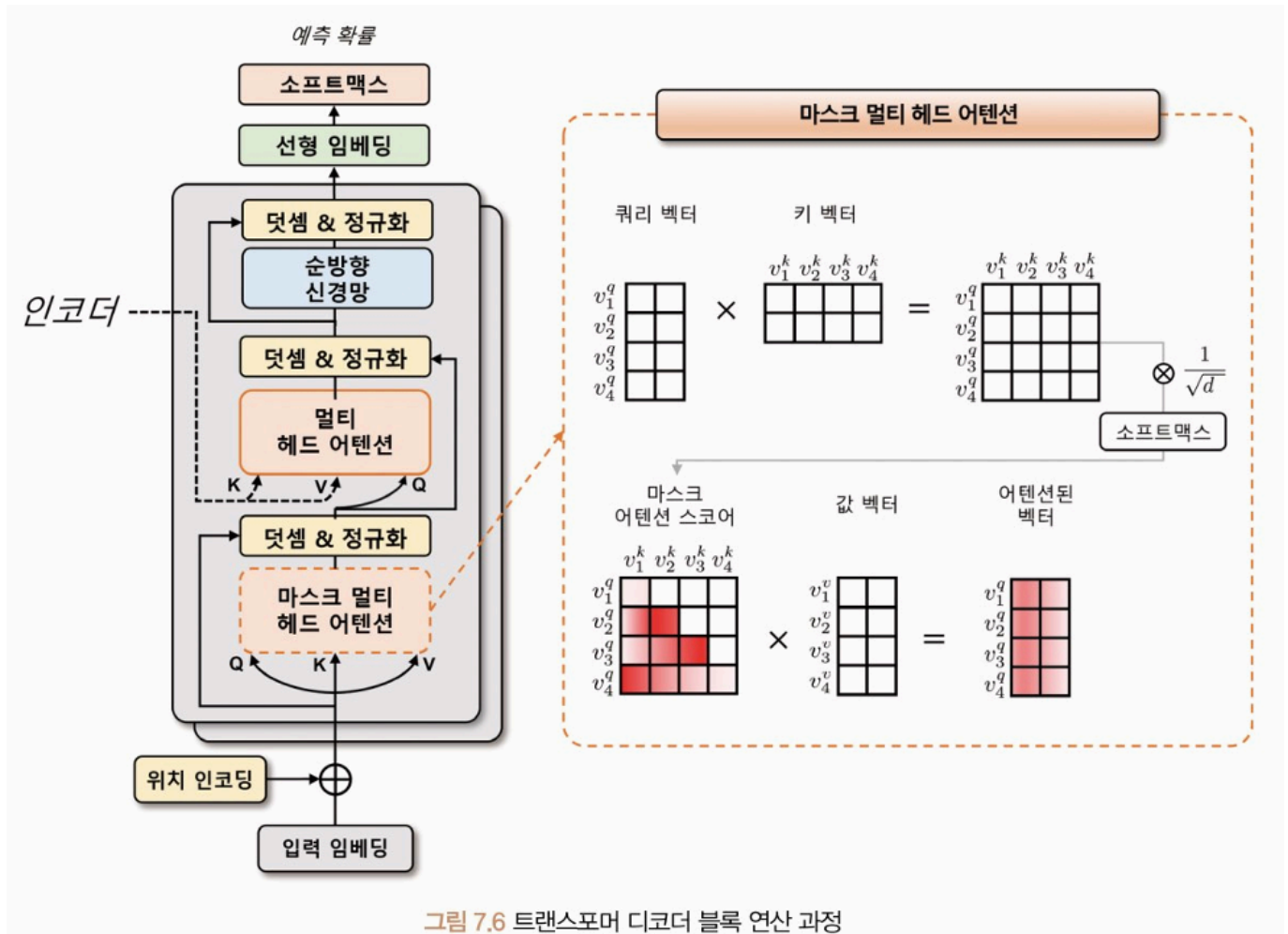
수식 7.2 어텐션 스코어 계산식

$$score(v^q, v^k) = \text{softmax}\left(\frac{(v^q)^T \cdot v^k}{\sqrt{d}}\right)$$

- 쿼리, 키, 값 벡터를 내적해 어텐션 스코어를 구함
- 구한 스코어값에 벡터 차원의 제곱근만큼 나눠 보정
- 보정된 어텐션 스코어를 소프트맥스 함수를 이용하여 확률적으로 재표현, 이를 값 벡터와 내적하여 셀프 어텐션된 벡터를 생성
- 멀티 헤드: 셀프 어텐션을 여러 번 수행해 여러 개의 헤드를 만듦
- 마지막 인코더 블록에서 출력된 벡터는 디코더에서 사용

트랜스포머 디코더: 위치 인코딩이 적용된 타겟 데이터의 입력 임베딩을 입력 받음

- 인코더의 멀티 헤드 어텐션 모듈은 인과성을 반영한 마스크 멀티 헤드 어텐션 모듈로 대체
 - 마스크 멀티 헤드 어텐션 모듈: 어텐션 스코어 맵을 계산할 때 첫 번째 쿼리 벡터가 첫 번째 키 벡터만을 바라볼 수 있게 마스크를 씌움
 - 마스크 영역에 수치적으로 작은 값인 -inf 마스크를 더해줌



- 멀티 헤드 어텐션에서 타깃 데이터가 쿼리 벡터로 사용, 인코더의 소스 데이터가 키와 값 벡터
- 쿼리 벡터, 키 벡터, 값 벡터를 이용해 어텐션 스코어 맵 계산, 소프트맥스 함수 적용하여 어텐션 가중치 구함
- 어텐션 가중치와 값 벡터를 가중합하여 출력 벡터를 얻음
- 트랜스포머의 디코더는 여러 개의 블록으로 구성
 - 이전 블록의 산출물은 다음 블록의 입력으로 전달
- 디코더는 타깃 데이터를 추론할 때 토큰/단어를 순차적으로 생성
 - PAD 토큰: 입력 토큰을 수차적으로 나타냄

각종 라이브러리

- 토치 데이터 라이브러리: 대규모 데이터셋을 다루기 쉽게 데이터를 불러오고 변환 및 배치하는 간단한 API 제공

- 토치 텍스트 라이브러리: 파이토치를 위한 텍스트 처리 라이브러리
- 포트락커 라이브러리: 파이썬에서 파일 락을 관리하기 위함

트랜스포머 클래스

- d_model: 트랜스포머 모델의 입력과 출력 차원의 크기를 정의
- nhead: 멀티 헤드 어텐션의 헤드의 개수
- num_encoder_layers/num_decoder_layers: 인코더와 디코더의 계층 수
- dim_feedforward: 순방향 신경망의 은닉층 크기
- dropout: 인코더와 디코더 계층에 적용되는 드롭아웃 비율
- activation: 순방향 신경망에 적용되는 활성화 함수
- layer_norm_eps: 계층 정규화를 수행할 때 분모에 더해지는 입실론 값

트랜스포머 순방향 메서드

- src/tgt: 인코더와 디코더에 대한 시퀀스
- src_mask/tgt_mask: 소스와 타겟 시퀀스의 마스크
- memory_mask: 인코더 출력의 마스크
- key_padding_mask: 소스, 타겟, 메모리 시퀀스에 대한 패딩 마스크

모델 번역 방식

- 그리드 디코딩: 디코더 네트워크가 생성한 확률 분포에서 가장 높은 확률을 가지는 단어를 선택하는 방법

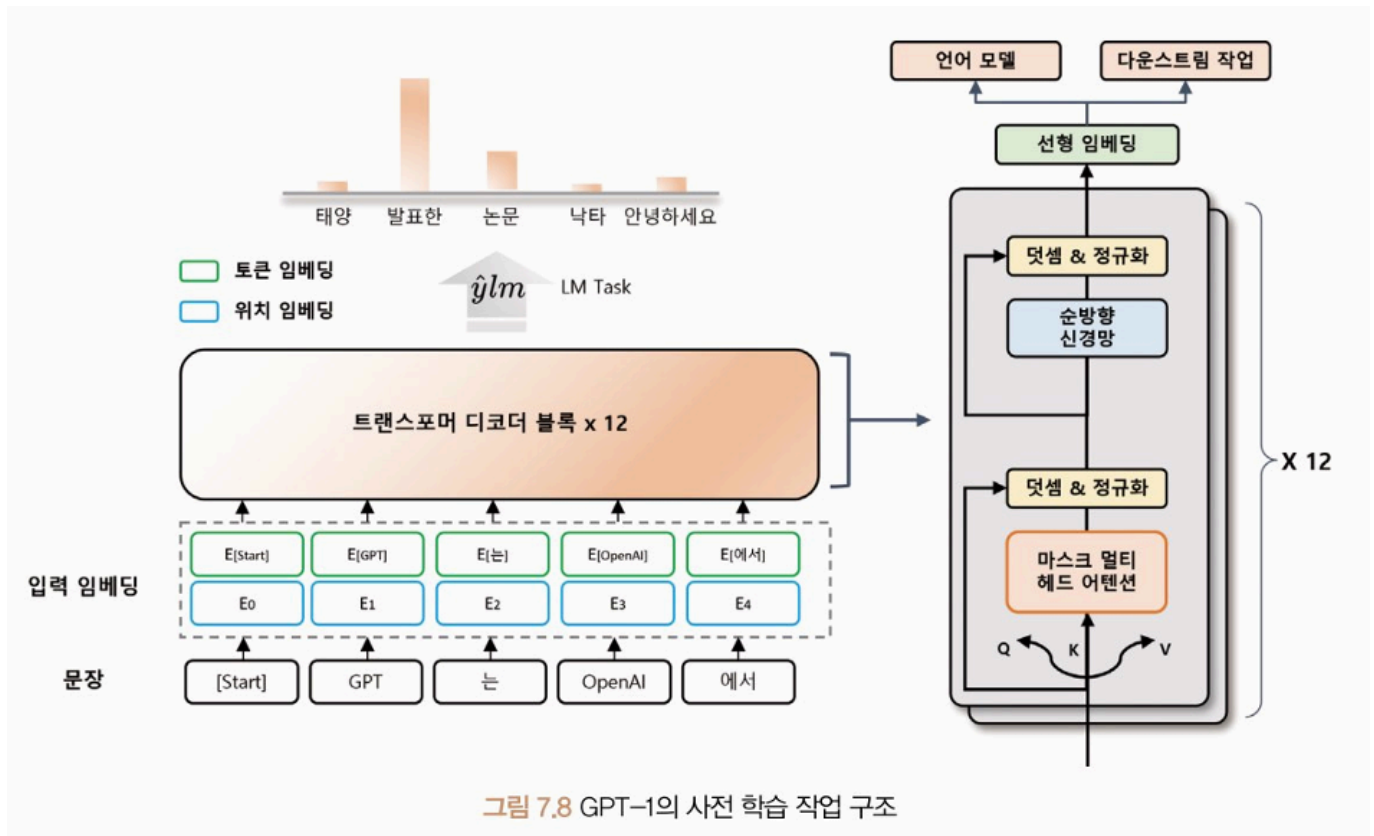
✓ GPT

GPT: 트랜스포머의 디코더를 여러 층 쌓아 만든 언어 모델

- 대규모 텍스트 데이터셋으로 사전 학습해 초기화, 특정 자연어 처리 작업에 맞게 미세 조정
- GPT-3.5: RLHF 방법을 도입해 모델을 학습
 - RLHF: 인간의 피드백을 이용하는 강화 학습 방법
- GPT-4: 이미지 데이터도 처리할 수 있게 개선

GPT-1: 트랜스포머 구조를 바탕으로 한 단방향 언어 모델

- 현재 위치에서 이전 단어들에 대한 정보만을 참고해 다음 단어를 예측
- 트랜스포머와 GPT-1 모델 비교: 인코더 사용 X, 디코더 부분만 사용, 디코더의 두 번째 다중헤드 어텐션 이용 X
- 약 4.5GB의 BookCorpus 데이터셋을 사용해 언어 모델을 사전 학습
 - 임의의 위치까지 보여줌, 뒤에 이어지는 문장을 모델이 자동으로 예측
 - 별도의 레이블이 필요하지 않아서 비지도 학습으로 학습이 이루어짐



- 사전 학습 구조
 - 입력 문장의 일부만 보고 다음 토큰을 예측하도록 하는 언어 모델을 통해 사전 학습
 - 입력 문장을 일정한 길이의 블록으로 나누어 처리, 각 블록에서는 블록 내의 첫 번째 토큰부터 순서대로 다음 토큰을 예측하게 함
- 입력 임베딩은 토큰 임베딩과 위치 임베딩을 이용해 계산
 - 토큰 임베딩: 각 토큰을 고정된 차원의 벡터로 매핑
 - 위치 임베딩: 입력 문장에서 각 토큰의 위치 정보를 나타내는 벡터를 매핑
- 미세 조정을 위한 다운스트림 작업에서 보조 학습해 사용 가능
 - 보조 학습: 주어진 학습 외에도 다른 작은 부가적인 학습 작업을 동시에 수행
 - 일반적인 언어 모델 학습 시 사용되는 손실 함수 + 추가 손실 함수 필요
 - 특수 토큰 사용

GPT-2: 1024의 길이까지 입력 처리 가능

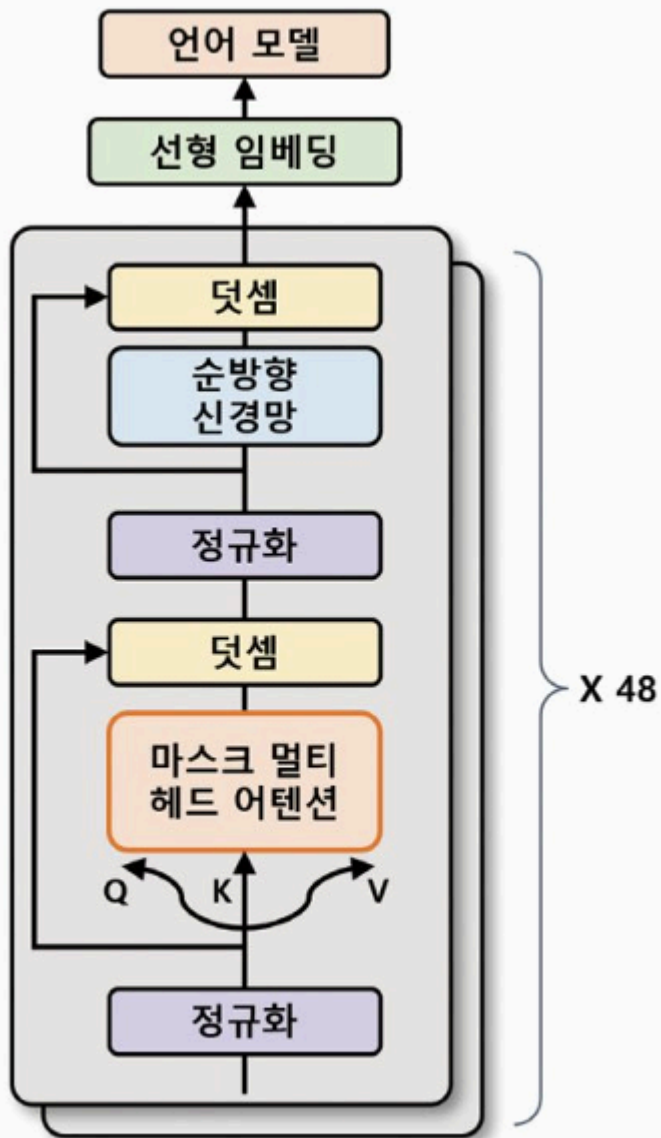


그림 7.10 GPT-2의 모델 구조

- GPT-1과의 비교
 - 12개의 디코더 층 -> 48개의 디코더 층
 - 1200만 개의 매개변수 -> 15억개의 매개변수
- 약 8백만 개의 문서를 사용해 약 40GB 데이터로 사전 학습 수행
- 제로-샷 학습 가능, 별도의 미세 조정 없이 다운스트림 작업에서 사용

GPT-3: 96개의 헤드를 가진 멀티 헤드 어텐션 사용, 96개의 디코더 층

- 멀티 헤드 어텐션 과정에서 희소 어텐션과 일반 어텐션을 섞어서 사용
- 토큰 임베딩 크기 1600 -> 12888
- 웹 크롤링, 위키백과, 서적 등에서 수집한 약 45TB에 달하는 방대한 양의 데이터세트를 이용
- 프롬프트 형식으로 작업 수행
- 새로운 도메인에서의 작업에 대해 높은 일반화 능력을 보임, 기존 자연어 처리 분야의 기술적 한계 극복

- BUT, 매우 느리고 비용이 많이 듦, 인간적인 이해력과 상식적인 추론 능력이 부족

GPT-3.5: GPT-3의 모델 구조를 따르면서도 RLHF 도입

- RLHF 학습 방법: 지도 미세 조정
 - 임의의 프롬프트를 가져오고 사람이 직접 적절한 답 작성
 - 작성된 답변을 모델이 생성한 문장과 함께 평가, 모델이 생성한 문장이 자연스러우면 보상을 받음
 - 보상을 바탕으로 강화 학습을 통해 모델을 학습
- 보상 모델을 학습, 시드에 따라 다른 답변을 생성하므로 내용이 다른 여러 개의 답변 생성 가능
 - 보상 모델로 사람이 평가하고 그에 따른 랭킹 부여
- 환각 현상이 존재: 모델이 인간과 같은 추론 능력을 갖추지 못하고 오류가 많은 답변 제시

GPT-4: 이미지 데이터도 인식 가능한 멀티모달 모델

- 최대 32768개의 토큰 입력 가능